Lightweight Post-Quantum Secure Communication Protocol for IoT Devices Using Code-Based Cryptography

Salahaldeen Duraibi and Abdullah Mujawib Alashjaee

***Abstract*—As the era of the Internet of Things (IoT) becomes more active, a large amount of information, including personal data, is being transmitted through IoT devices. To protect this information, it is important for devices to communicate using mutual encryption. Due to the performance limitations inherent in IoT devices, lightweight security protocols are required. Currently, most cryptographic techniques used in security protocols are based on RSA and ECC (Elliptic Curve Cryptography). However, if high-performance quantum computers are developed and Shor's algorithm is utilized, the foundational security assumptions of RSA and ECC can be easily broken, rendering them unusable. Therefore, this article proposes a security protocol that is resilient to the computational capabilities of quantum computers. It employs ROLLO, a code-based cryptographic algorithm that is currently under consideration in the NIST (National Institute of Standards and Technology) post-quantum cryptography standardization competition. Additionally, to facilitate mutual communication between IoT devices, computationally efficient operations such as hashing and XOR are utilized. Finally, the proposed protocol is compared and analyzed in terms of performance and security against existing protocols.**

***Index Terms*—IoT security, lightweight cryptography, post-quantum cryptography, ROLLO, code-based encryption, mutual authentication, hash function, XOR operation, Shor's algorithm, quantum-resistant protocol.**

I. Introduction

With the rapid proliferation of the Internet of Things (IoT), its applications have expanded across numerous domains, including healthcare, smart cities, agriculture, and industrial automation [1]. However, the integration of IoT into critical systems introduces significant privacy and security concerns, especially in scenarios involving sensitive data. For instance, in an IoT-based healthcare framework, a malicious actor could inject falsified medical data, potentially leading to erroneous diagnoses or treatments by healthcare professionals [2], [3]. This exemplifies the critical need for robust security mechanisms that can safeguard user data and ensure trustworthiness in communication.

To address such security challenges, it is essential to develop lightweight cryptographic protocols tailored for resource-constrained IoT devices. Traditional security mechanisms often impose substantial computational and energy burdens, which are impractical for IoT nodes. Hence, there is a pressing need for efficient cryptographic solutions that balance strong security with low computational overhead [4].

Presently, Elliptic Curve Cryptography (ECC) is one of the most popular public-key cryptosystems employed in IoT due to its efficiency and smaller key sizes. For example, Xiao et al. [5] proposed an ECC-based protocol in 2025 that exemplifies lightweight cryptographic design [6]. Nonetheless, ECC's security hinges on the hardness of the Elliptic Curve Discrete Logarithm Problem (ECDLP), which is vulnerable to quantum attacks, particularly Shor's algorithm, thereby rendering it insecure in the quantum computing era [7].

The growing reliance on Internet of Things (IoT) devices has intensified the need for secure and lightweight post-quantum cryptographic (PQC) [8] solutions tailored for constrained environments. Kumar et al. [9] proposed a PQC-based multimedia encryption scheme incorporating chaos maps and Number Theory Research Unit (NTRU)-based cryptography to secure communication in consumer electronics. Similarly, Sharma and Rani [10] developed a hybrid cryptographic framework that integrates post-quantum cryptography with deep learning for intrusion detection in IoT consumer devices, enhancing adaptive defense mechanisms against sophisticated threats.

Several studies focus on enhancing data privacy and authentication through advanced cryptographic primitives [11]. For instance, Peelam and Chamola [12] introduced a quantum blockchain approach that leverages quantum key distribution (QKD) to secure distributed ledgers in consumer IoT networks. Huang and Sharma [13] proposed a post-quantum verifiable decryption scheme utilizing lattice-based group authentication and signcryption mechanisms, which offer robust resistance against quantum adversaries on edge-based consumer devices.

Efforts have also been made to design lightweight, energy-efficient post-quantum authentication protocols [14], [15]. Seno et al. [16] introduced a blind signature scheme based on CRYSTALS-Dilithium and lattice-based techniques to secure constrained IoT environments with minimal computational costs. Yao et al. [17] proposed a multi-protocol QKD

Received 31 May 2025; revised 3 August 2025 and 31 August 2025; accepted 14 September 2025. Date of publication 18 September 2025; date of current version 8 December 2025. This work was supported by the Deanship of Scientific Research at Northern Border University, Arar, KSA, under Project NBU-FPEJ-2025-1576-02. *(Corresponding author: Abdullah Mujawib Alashjaee.)*

Salahaldeen Duraibi is with the Department of Electrical and Electronics Engineering, and the College of Engineering and Computer Science, Jazan University, Jazan 45142, Saudi Arabia (e-mail: sduraibi@jazanu.edu.sa).

Abdullah Mujawib Alashjaee is with the Department of Computer Sciences, and the College of Science, Northern Border University, Arar 91431, Saudi Arabia (e-mail: abdullah.alashjaee@nbu.edu.sa).

Digital Object Identifier 10.1109/TCE.2025.3611586

networking scheme aimed at supporting secure communication across diverse consumer electronics with collaborative protocol integration. Castiglione et al. [18] presented an integration of PQC and blockchain for securing microcontroller-based low-cost IoT devices using Dilithium-5 digital signatures.

From a performance optimization perspective, Althobaiti and Dohler [19] designed a location-based lattice cryptography scheme for the Narrowband IoT ecosystem. Hafeez et al. [20] developed GPU-accelerated TMVP-based polynomial convolution algorithms to increase throughput in PQC systems targeting IoT. Ahmad and Jagatheswari [21] proposed a lattice-based three-party key agreement scheme for medical IoT applications, focusing on reducing handshake overhead while maintaining post-quantum resilience. Shim [22] systematically assessed the energy consumption, fault resilience, and implementation efficiency of various post-quantum signature schemes under NIST guidelines for IoT devices.

In big IoT networks, scalability means that a system can expand and manage more connected devices and the vast amounts of data they produce without sacrificing efficiency, dependability, or affordability. Using cloud computing for elastic resources, creating modular and microservices-based platforms, utilizing software-defined networking (SDN) for dynamic resource allocation, and putting in place reliable data management systems that can handle big datasets are all crucial to doing this. A major issue during the shift to quantum-safe technologies is backward compatibility with non-quantum-safe equipment. Hybrid encryption models and transitional solutions, such as symmetric key wrappers or post-quantum certificate replacements, solve this issue. To ensure secure communication between quantum-safe and non-quantum-safe systems, hybrid techniques include classical and post-quantum algorithms.

Consequently, significant research has shifted towards post-quantum cryptography (PQC), which aims to develop cryptographic algorithms resistant to quantum computational capabilities [6]. The U.S. National Institute of Standards and Technology (NIST) initiated a call for standardizing PQC algorithms in 2016–2017, leading to a global response and the submission of numerous algorithm candidates. Among these, 32 have progressed to Round 2, encompassing cryptographic constructions based on lattices, codes, multivariate polynomials, and isogeny-based approaches.

This article focuses on addressing the need for a lightweight, post-quantum secure communication protocol suitable for IoT environments. Specifically, we aim to:

- Design a secure communication protocol using ROLLO—a code-based cryptographic scheme currently under NIST evaluation.
- Ensure lightweight implementation by employing simple and efficient operations such as hashing and XOR, thereby reducing encryption and decryption overhead.
- Evaluate and compare the security of the proposed protocol against various attack vectors.
- Perform an empirical performance analysis in terms of key generation, encryption, and decryption timings, and benchmark it against other state-of-the-art code-based cryptographic schemes.

Our contributions offer practical insights into the feasibility of integrating post-quantum cryptographic solutions within constrained IoT devices, enhancing their resilience to both classical and quantum threats.

II. Related Work

A. Code-Based Cryptography

The principle of code-based cryptography involves the sender intentionally attaching correctable errors to a message. The legitimate receiver, who knows the error-correcting code, can easily correct these errors. In 1978, Robert J. McEliece proposed the first code-based cryptosystem, known as McEliece [23].

McEliece uses an error-correcting code called the Goppa code [24]. Among the seven code-based cryptosystems in Round 2 of the NIST post-quantum cryptography competition, Classic McEliece and NTS-KEM still use Goppa codes directly. While Goppa codes boast a long history and strong security, they suffer from the drawback of extremely large key sizes.

To overcome this limitation, recent research focuses on using new codes instead of Goppa codes, aiming to reduce key sizes. Five of the Round 2 candidates are based on alternative structures such as Quasi-Cyclic and Rank Metric codes [25].

B. NIST PQC Competition Code-Based Schemes and ROLLO

ROLLO [26] is one of the code-based cryptographic schemes currently in Round 2 of the NIST post-quantum cryptography standardization process. Unlike Goppa code-based schemes, ROLLO prioritizes efficiency by using a Rank Metric code (introduced in 1990's), offering significant advantages in terms of key size and computational complexity [27].

NIST is now on its second round of standards selection for post-quantum cryptography. When choosing the next standards, NIST views the algorithms' performance and cost as the second most crucial factor. Because of security flaws discovered in a key recovery attack, the ROLLO cryptosystem was deemed a concern and was not chosen for additional assessment in NIST's PQC Round 2. ROLLO was a collection of code-based candidates for post-quantum cryptography, but after the first screening and before the second round of thorough examination started, a major assault solved its fundamental mathematical issue, which resulted in its removal from the standardization process.

To compare its performance with other Goppa code-based schemes such as Classic McEliece and NTS-KEM, the execution speed of all seven code-based candidates was measured on a low-power mobile processor (ARM). The experimental environment and the speed comparison results are presented in Table I and Table II.

All cryptographic schemes compared, including LEDAcrypt, Classic McEliece, and ROLLO, operate at the 128-bit post-quantum security level as defined by the NIST PQC standardization process. This ensures a fair comparison across performance metrics.

A comparative analysis of the characteristics and performance of the seven code-based cryptography candidates

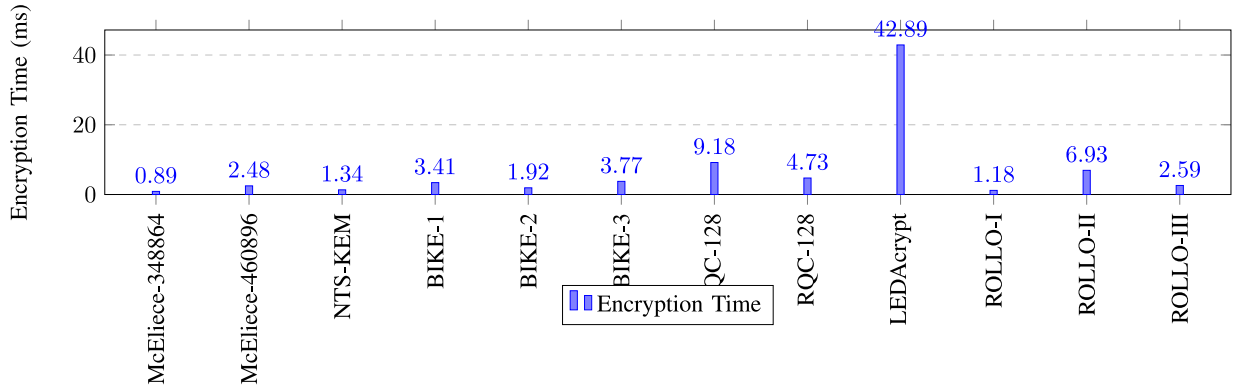

Fig. 1. Encryption time comparison (in ms) among code-based cryptographic schemes from NIST Round 2.

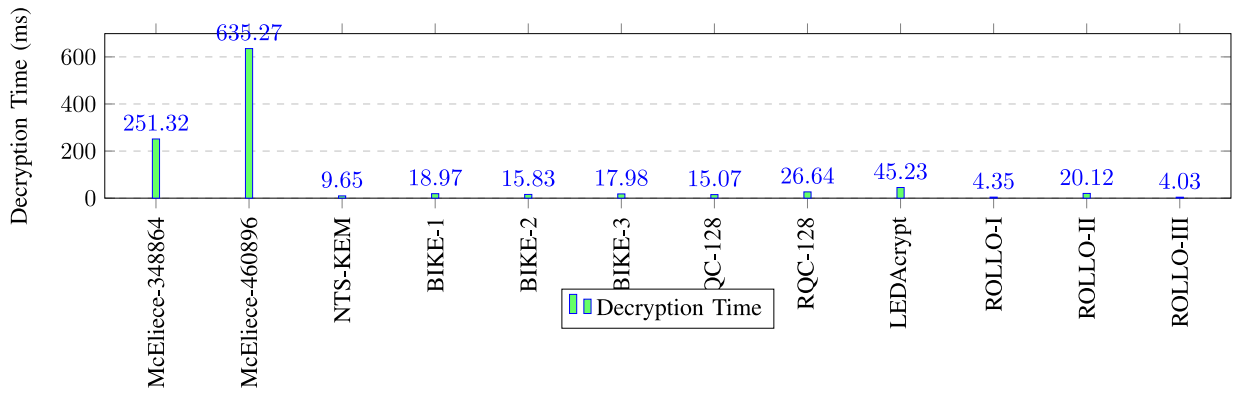

Fig. 2. Decryption time comparison (in ms) among code-based cryptographic schemes from NIST Round 2.

TABLE I
EXPERIMENT ENVIRONMENT

Component	Specification
Platform	NVIDIA Jetson Nano
CPU	Quad-core ARM Cortex-A57 @ 1.43 GHz
Memory	4 GB LPDDR4
Operating System	Ubuntu 20.04 (JetPack SDK)

TABLE II
COMPARISON OF TIMINGS (IN MILLISECONDS) FOR KEY MANAGEMENT SCHEMES

Scheme	Key Generation	Encryption	Decryption
Classic McEliece Variants			
Classic McEliece-348864	1785.12	0.89	251.32
Classic McEliece-460896	3859.78	2.48	635.27
NTS-KEM and BIKE Variants			
NTS-KEM Level-1	294.30	1.34	9.65
BIKE-1 (CCA-secure)	3.52	3.41	18.97
BIKE-2 (CCA-secure)	38.42	1.92	15.83
BIKE-3 (CCA-secure)	1.95	3.77	17.98
HQC, RQC, and LEDAcrypt			
HQC-128 (1st variant)	51.02	9.18	15.07
RQC-128	2.58	4.73	26.64
LEDAcrypt PKE	4128.47	42.89	45.23
ROLLO Variants			
ROLLO-I-128	8.32	1.18	4.35
ROLLO-II-128	74.81	6.93	20.12
ROLLO-III-128	1.70	2.59	4.03

reveals two groups: those that continue to use traditional Goppa codes, such as Classic McEliece and NTS-KEM, and those that employ new codes, such as the Rank Metric code used in ROLLO.

Goppa codes, with a 45-year history, are known for their strong security, but they suffer from inefficiency. In contrast, newly researched codes have a relatively short verification period but demonstrate high performance. At the 128-bit security level, Classic McEliece generates a ciphertext of only 128 bytes, but its public key size is 261,120 bytes and the private key size is 6,452 bytes—sizes that are too large to be stored on low-performance 8-bit AVR processors. On the other hand, the ROLLO-II scheme used in this article offers a much smaller key size, with a public key of 1,941 bytes, a private key of 40 bytes, and a ciphertext size of 2,089 bytes.

Initially, all seven code-based schemes provided security at the IND-CPA level. However, as the competition progressed to Round 2, most schemes transitioned from a traditional Public Key Encryption (PKE) structure to a Key Encapsulation Mechanism (KEM) structure, thereby upgrading their security to the IND-CCA level. In a KEM structure, the actual message is encrypted using a symmetric key, which is shared using public key encryption—a hybrid cryptographic approach. Currently, among the Round 2 code-based schemes, only LEDAcrypt and ROLLO provide PKE versions.

In this article, we selected ROLLO-II as the encryption mechanism for the proposed protocol. ROLLO-I and ROLLO-III operate under the KEM framework, where public and private keys are used to secretly establish a symmetric key between both parties. This symmetric key is then used to encrypt and decrypt messages. However, ROLLO-II provides a PKE structure, allowing direct encryption of messages using

Algorithm 1 Device Registration Phase (Author's Protocol)

Require: Device ID ID_D, Server ID ID_S, Server Public Key PK_S, Device Public Key PK_D, Device Private Key SK_D, Server Private Key SK_S
Ensure: Registered device with password securely stored on device and server
Step 1: Registration Request
Device generates nonce N_1 and timestamp ts
$REQ \leftarrow \text{Enc}_{PK_S}(ID_D \| N_1 \| ts)$
Send REQ to Server
Step 2: Server Response
Server decrypts REQ: $(ID_D, N_1, ts) \leftarrow \text{Dec}_{SK_S}(REQ)$
Server verifies timestamp ts
Server generates password Pw and temporary password Pwt
$et_1 \leftarrow \text{Enc}_{PK_D}(N_1 \| Pw)$
Send et_1 to Device
Step 3: Device Password Verification
Device decrypts et_1: $(N_1', Pw) \leftarrow \text{Dec}_{SK_D}(et_1)$
if $N_1' = N_1$ **then**
    Store Pw
else
    Abort registration
end if
Step 4: Device Confirmation
$\rho_1 \leftarrow H(Pw \oplus N_1)$
Send ρ_1 to Server
Step 5: Server Finalization
Server verifies ρ_1
if valid **then**
    Store (ID_D, Pw) in database
    Acknowledge successful registration
else
    Abort registration
end if
return Device is securely registered with the server

Algorithm 2 Device Authentication Phase (Author's Protocol)

Require: Device ID ID_D, Password Pw, Temporary Password Pwt, Server Public Key PK_S, Device Public Key PK_D, Device Private Key SK_D, Server Private Key SK_S
Ensure: Mutual authentication and session key establishment
Step 1: Device Request
Device generates nonce N_2 and timestamp ts
$et_2 \leftarrow \text{Enc}_{PK_S}(ID_D \| N_2 \| ts)$
$\rho_2 \leftarrow H(Pw \oplus N_2)$
Send (et_2, ρ_2) to Server
Step 2: Server Verification
Server decrypts et_2: $(ID_D', N_2, ts) \leftarrow \text{Dec}_{SK_S}(et_2)$
Verify timestamp ts
Retrieve Pw and Pwt from database
if $\rho_2 = H(Pw \oplus N_2)$ or $\rho_2 = H(Pwt \oplus N_2)$ **then**
    Update password: $Pw_n \leftarrow Pw \oplus N_2$
    $\rho_3 \leftarrow H(Pw_n)$
    Send ρ_3 to Device
else
    Abort authentication
end if
Step 3: Device Verification
if $\rho_3 = H(Pw_n)$ **then**
    Update password: $Pw \leftarrow Pw_n$
    $\rho_4 \leftarrow H(Pw_n \oplus ID_D)$
    Send ρ_4 to Server
    $SK \leftarrow H(Pw_n \| ID_D)$
else
    Abort authentication
end if
Step 4: Server Finalization
if $\rho_4 = H(Pw_n \oplus ID_D)$ **then**
    Delete Pwt from database
    $SK \leftarrow H(Pw_n \| ID_D)$
    Authentication successful
else
    Abort authentication
end if
return Shared session key SK established

the public key and decryption using the private key. Although ROLLO-I and ROLLO-III could also be used, this article opts for ROLLO-II due to its suitability for directly encrypting and transmitting device IDs, passwords, and nonce values.

C. Security Protocol

In 2022, author proposed a lightweight security protocol [28] using a variant of the code-based cryptosystem McEliece, known as Niederreiter. The protocol consists of two procedures: registration and authentication, as shown in Algorithms 1 and 2. A description of the notations used can be found in Table III.

In terms of security, the protocol includes timestamps and nonce values in the messages to prevent replay attacks during communication. For device anonymity, the device ID_D is encrypted using the code-based Niederreiter cryptosystem during both the registration and communication phases. Furthermore, all messages exchanged between devices and the server are encrypted. As a result, the registration phase requires three encryptions and decryptions, and the authentication phase also involves three encryptions and decryptions [29].

In the protocol proposed in this article, the registration phase—where initial identity verification and password assignment occur—follows the same structure, involving two encryptions. However, in subsequent communication, only the previously used nonce, along with hash and XOR operations, are employed. These are far simpler than cryptographic computations, greatly reducing the computational burden on both devices and the server.

During authentication, only one encryption is used at the beginning to preserve device anonymity. Afterward, secure communication is maintained using only nonce values, hash functions, and XOR operations. A detailed comparison of the proposed protocol with existing protocols in terms of performance and security is presented in Section VI.

III. Proposed Protocol

The proposed security protocol consists of two main components: a registration procedure for legitimate devices with the server, and an authentication procedure that enables registered devices to communicate securely with the server. ROLLO-I was created with IND-CPA security in mind, which is a less robust security paradigm than CCA2. Its reduced public key sizes and lower RAM requirements made it more appropriate for places with limited resources, such as microcontrollers. Targeting IND-CCA2 security, ROLLO-II was created for a better level of protection. Its enhanced security made it more appropriate for a wider range of applications, even if it had larger parameters and might have used more RAM than ROLLO-I. Although the snippets provided provide less detail on its unique variations, ROLLO-III was also included in the ROLLO submittal. Because it offered a different set of security and performance features, ROLLO-II was chosen to give the NIST process more alternatives within the ROLLO framework.

Algorithm 3 Authentication Phase Using ROLLO-II

Require: Device ID ID_D, Password Pw, Server Public Key PK_S, Device Private Key SK_D
Ensure: Mutual authentication and session key establishment between device and server
Step 1: Device Initiation
Device generates nonce N_2
$et_2 \leftarrow \text{Enc}_{PK_S}(ID_D \| N_2)$
$\rho_2 \leftarrow H(Pw \oplus N_2)$
Send (et_2, ρ_2) to Server
Step 2: Server Verification
Server decrypts et_2: $(ID'_D, N_2) \leftarrow \text{Dec}_{SK_S}(et_2)$
Retrieve Pw from database using ID'_D
if $\rho_2 = H(Pw \oplus N_2)$ **then**
  Generate new password $Pw_n \leftarrow Pw \oplus N_2$
  $\rho_3 \leftarrow H(Pw_n)$
  Send ρ_3 to Device
else
  Abort authentication
end if
Step 3: Device Response
if $\rho_3 = H(Pw \oplus N_2)$ **then**
  Update $Pw \leftarrow Pw_n$
  $\rho_4 \leftarrow H(Pw_n \oplus ID_D)$
  Send ρ_4 to Server
  $SK \leftarrow H(Pw_n \| ID_D)$
else
  Abort authentication
end if
Step 4: Server Finalization
if $\rho_4 = H(Pw_n \oplus ID_D)$ **then**
  Delete temporary password
  $SK \leftarrow H(Pw_n \| ID_D)$
  Authentication successful
else
  Abort authentication
end if
return Shared session key SK established between device and server

For the encryption mechanism, the protocol employs ROLLO-II, one of the code-based cryptosystems (ROLLO-I, II, III), which is designed to be resistant to quantum computer attacks.

Among the operations used in the protocol, encryption and decryption are the most computationally expensive. Therefore, to ensure lightweight design, the proposed protocol minimizes the number of encryption and decryption operations. All other operations rely only on the SHA-512 hash function and XOR operations.

The protocol leverages effective primitives like SHA-512 hash algorithms and XOR operations in place of costly cryptographic operations to preserve a lightweight design. Because of the substantial reduction in computing cost, the protocol can be used in real-time, energy-sensitive Internet of Things scenarios. The SHA-512 hash of the device ID and the revised password is used to generate the session key, which yields a 512-bit symmetric key with robust protection against brute-force attacks.

The notations used to describe the protocol are provided in Table III.

TABLE III
NOTATION

Symbol	Description
REQ	Registration request message
Enc_{PK}	Encryption using public key
Dec_{S_K}	Decryption using private key
ID_D	Device identifier
ID_S	Server identifier
N_1, N_2	Nonce values (random numbers)
t_s	Timestamp
P_w	Password
P_{wt}	Temporary password
$H(\cdot)$	Hash function
D_B	Database
S_K	Session key

Algorithm 4 Device Registration Phase Using ROLLO-II

Require: Device ID ID_D, Server Public Key PK_S, Device Private Key SK_D
Ensure: Securely registered device with password stored on server and device
Step 1: Registration Request
Device generates nonce N_1
$REQ \leftarrow \text{Enc}_{PK_S}(ID_D \| N_1)$
Send REQ to Server
Step 2: Server Response
Server decrypts REQ: $(ID_D, N_1) \leftarrow \text{Dec}_{SK_S}(REQ)$
Server generates temporary password Pw
$et_1 \leftarrow \text{Enc}_{PK_D}(N_1 \| Pw)$
Send et_1 to Device
Step 3: Device Verification
Device decrypts et_1: $(N'_1, Pw) \leftarrow \text{Dec}_{SK_D}(et_1)$
if $N'_1 = N_1$ **then**
  Store Pw
else
  Abort registration
end if
Step 4: Final Confirmation
$\rho_1 \leftarrow H(Pw \oplus N_1)$
Send ρ_1 to Server
Server verifies ρ_1 and stores (ID_D, Pw) in database
return Device successfully registered and ready for authentication

A. Device Registration

The device registration process consists of four steps, as illustrated in Algorithm 3.

1) First, the device wishing to register to the server encrypts its device ID_D and a nonce value using the server's public key and sends the encrypted message.
2) Second, the server decrypts the received request message REQ using its private key to verify the device ID_D and nonce value. Then, it generates a password for the device. The server encrypts the nonce value for authentication and the generated password with the device's public key and sends it back.
3) Third, upon receiving the ciphertext et_1, the device decrypts it using its private key. If the recovered nonce value matches, it safely stores the password.
4) Finally, the device computes $\rho_1 = h(P_w \odot N_1)$ and sends it to the server. Knowing the hash input, the server verifies ρ_1 and stores the device ID_D and password in its database.

B. Mutual Authentication Between Device and Server

The mutual authentication between the registered device and the server consists of four steps, as shown in Algorithm 4.

1) First, the device encrypts the nonce value it generated and its device ID_D using the server's public key to produce ciphertext et_2. It also computes $\rho_2 = h(P_w \odot N_2)$

using the password assigned at registration and a new nonce value N_2, then sends both et_2 and ρ_2 to the server.
2) Upon receiving, the server decrypts et_2 using its private key to recover the device ID_D, then compares it with its database to retrieve the device's password and temporary password. If the hash value ρ_2 is verified using the nonce and either password, the server replaces the old password with the temporary one and updates the password as $P_{wn} = P_w N_2$. It then sends the hash of the new password back to the device.
3) The device verifies the received hash ρ_3 using its password and the nonce it generated. After successful verification, it updates its password similarly, computes $\rho_4 = h(P_{wn} \odot ID_{\text{id}})$, sends ρ_4 to the server, and establishes a session key $S_K = h(P_{wn} \mid ID_{\text{id}})$.
4) Finally, the server constructs the same hash input and verifies ρ_4. If verified, the server deletes the temporary password and establishes the session key matching the device.

The session key is derived using the SHA-512 hash of the updated password and device ID, resulting in a 512-bit symmetric key, which offers strong security against brute-force attacks. A cryptographically secure random number generator (CSPRNG) is used to produce all nonces and random values in order to prevent entropy reuse and guarantee unpredictability. Using a CSPRNG, each device creates a unique nonce per session to avoid once reuse. Devices can actually employ a counter or date to determine the last-used nonce or track it to guarantee uniqueness between sessions, even after rebooting. They are generally reseeded on a regular basis to maintain their unpredictability after being initialized with a high-quality seed from an entropy source (such as hardware or system noise). All nonces and random values are generated using a cryptographically secure random number generator (CSPRNG) to ensure unpredictability and avoid entropy reuse. The session key $SK = H(P_{wn} \| ID_D)$ is **re-derived in every session** using a newly updated password P_{wn} derived from a fresh nonce. This ensures that the key is **ephemeral and unique per session**, rather than being reused or persisted, thereby supporting *Perfect Forward Secrecy (PFS)* and preventing linkage across sessions.

IV. Security Analysis

A. Adversarial Model

The proposed protocol is analyzed under the Dolev-Yao model, where the adversary has full control over the communication channel—able to intercept, modify, and inject messages. We also assume a quantum-capable adversary, able to execute attacks like Shor's algorithm on RSA/ECC but unable to efficiently break code-based cryptography (e.g., ROLLO).

The adversary may attempt replay, impersonation, MITM, and session key compromise. However, due to the use of nonces, hash-based verification, and ephemeral password updates, the protocol resists these threats while ensuring device anonymity, message integrity, and Perfect Forward Secrecy (PFS).

B. Device Anonymity and Information Confidentiality

In the proposed protocol, the device ID_D which can reveal the device's identity is encrypted at the registration and mutual authentication stages. Because the ciphertext changes every time due to the nonce values, it is impossible to trace which device is communicating at any time. Moreover, only authorized users can access transmitted information. The proposed protocol encrypts critical information such as device ID_D, nonce values, and passwords through cryptographic algorithms, hash functions, and XOR operations, preventing unauthorized users from accessing this information. To prevent nonce reuse, each device generates a fresh nonce per session using a cryptographically secure random number generator (CSPRNG). In practice, devices may track the last-used nonce or derive it from a counter or timestamp to ensure uniqueness across sessions, even after reboot.

Nonce values are typically disposed of after a single usage or a limited lifespan in order to preserve security and stop replay attacks. Never repeat a nonce in cryptographic applications since this renders the security it was designed to offer null and void. A nonce, as used in cryptography, is a random or pseudo-random number that needs to be distinct for every encryption or authentication procedure. Reusing a nonce weakens the encryption and leaves it open to intrusion.

C. Man-in-the-Middle and Replay Attacks

The proposed protocol verifies the authenticity of all communication through authentication messages ρ during both device registration and session key establishment, ensuring security against man-in-the-middle attacks. For replay attacks, many protocols use nonce values or timestamps. However, timestamps require synchronization between communicating parties. Therefore, the proposed protocol uses nonce values. Since the nonce generated at the session start is encrypted and affects the hash inputs of all transmitted messages, every message is tied to the nonce. Thus, if a message from a previous session is resent, it is detected as a replay attack and prevented.Although the protocol omits timestamps to reduce synchronization overhead, the use of session-unique nonces ensures freshness of each communication. Since nonces are never reused and are cryptographically tied to hash computations in each session, they are sufficient to detect and prevent replay attacks—even in long-duration sessions—provided that nonces are securely generated and session-specific.

D. Perfect Forward Secrecy (PFS)

To achieve PFS, the security of past communication records must be maintained even if the private key is exposed. In the proposed protocol, even if the device is compromised and the password is exposed, previous passwords cannot be derived without knowing the nonce values. Therefore, past communication records cannot be decrypted, ultimately achieving PFS. While the protocol achieves Perfect Forward Secrecy through ephemeral password updates and nonce usage, a compromised public key would allow an adversary to impersonate a server or device. The current design assumes public key integrity is

TABLE IV
PARAMETERS FOR ROLLO-II AND NIEDERREITER

Method	Security level	Public key	Secret key
ROLLO-II	128-bit	1758 bytes	36 bytes
Niederreiter	128-bit	116,150 bytes	24,200 bytes

TABLE V
EXPERIMENT ENVIRONMENT

Component	Specification
CPU	Intel Core i9-12900K @ 5.2 GHz
Memory	64GB DDR5 RAM
OS	Ubuntu 22.04 LTS

maintained via a trusted initialization or certificate authority. However, no automatic key refresh is included. In future work, we aim to integrate a lightweight key update mechanism to mitigate long-term risks from public key exposure.

V. COMPARATIVE ANALYSIS

A. Protocol Performance

Author's protocol uses the Niederreiter cryptosystem based on Goppa codes for encryption. While Goppa codes provide fast encryption and decryption speeds, their key sizes are very large. The Niederreiter key size is so large that it cannot be stored on low-memory devices such as 8-bit AVR processors. In contrast, the ROLLO cryptosystem used in the proposed protocol offers a suitable key size and competitive computational speed.

While this study focuses on CPU execution time, RAM and stack usage were not profiled. Given the importance of memory constraints in IoT deployments, future work will include detailed memory footprint analysis on microcontroller-class platforms.

Table IV shows the key sizes of the Niederreiter cryptosystem at a 128-bit security level used in Author's protocol, and the key sizes of ROLLO-II at the same security level used in the proposed protocol. Classic McEliece only produces a 128-byte ciphertext at the 128-bit security level, but its public key and private key are too big to fit on low-performance 8-bit AVR processors, measuring 261,120 bytes and 6,452 bytes, respectively. The NIST PQC standardization procedure defines the 128-bit post-quantum security level at which all of the cryptographic algorithms under comparison—including LEDAcrypt, Classic McEliece, and ROLLO—operate. This makes sure that all performance indicators are fairly compared.

For performance evaluation of the proposed protocol, measurements were conducted on a low-power ARM processor commonly used in many IoT devices. Additionally, to verify communication speed on a typical computer, performance was also measured on a high-performance Intel processor. The ARM processor environment is described in Table I, and the Intel processor environment is shown in Table V. The results for these environments are presented in Tables VI and VII, respectively.

The Jetson Nano was chosen to reflect a realistic low-power IoT edge device capable of running post-quantum cryptography. The Intel i9 provides a performance upper bound. While MCU-class devices like Cortex-M4 have tighter constraints, the protocol's minimal encryption and use of hash/XOR operations suggest it can be adapted with optimization or hardware support. A cryptographic procedure known as a key derivation function (KDF) uses pseudorandom functions to generate one or more new cryptographic keys given a secret input, like a master key or password. By generating strong keys that are computationally identical to random strings, KDFs offer security, safeguarding data both during storage and transmission. Keyed hash functions known as Hash-Based Key Derivation Functions (HashKDFs) are useful for deterministically deriving several keys from a single root key. This might make your key management plans simpler. Benchmarking on such platforms is planned as future work. Since encryption requires significant computation, the number of encryptions performed is critical for designing a lightweight protocol. Encryption and decryption consume the largest portion of computational cost. A comparison of encryption counts is given in Table VIII.

TABLE VI
PERFORMANCE IN ARM PROCESSOR (IN MS)

Protocol	Registration	Authentication
Proposed Method	90 $\pm$3.1	45 $\pm$2.2

TABLE VII
PERFORMANCE IN INTEL PROCESSOR (IN MS)

Protocol	Registration	Authentication
Proposed Method	5 $\pm$0.4	2.5 $\pm$0.3

TABLE VIII
COMPARISON OF ENCRYPTION COUNT

Protocol	Registration	Authentication
Reference Method	5	5
Proposed Method	3	2

In author's protocol, encryption and decryption are performed 5 times during registration and 5 times during authentication. In contrast, the proposed protocol replaces most operations with hash functions and XOR operations, performing encryption only 3 times during registration and once during authentication. During registration, since identity verification is required initially, encryption is performed once by the device and once by the server, totaling 3 encryptions. In the authentication phase, encryption is performed only once for the password and the device ID_D to ensure anonymity, and subsequent authentication uses verified passwords and hash functions to prove identity.

B. Protocol Security

From the perspective of achieving Perfect Forward Secrecy (PFS), author's protocol uses passwords and private keys statically during the authentication process. Therefore, if a device is compromised and the private key and password are exposed, past communication records can be hacked. However, the proposed protocol updates the password using a nonce value after communication. Even if the device's private key is compromised, the password necessary to hack past

TABLE IX
COMPARISON OF EXISTING POST-QUANTUM CRYPTOGRAPHIC SCHEMES FOR IOT

Scheme	Quantum Resistance	Key Size (KB)	Encryption Time (ms)	Communication Overhead (Bytes)	Suitable for IoT
CRYSTALS-Kyber	High	1.5–2.6	2–3	∼1,500	Moderate
BIKE	High	1.2–2.8	3–5	∼1,300	Moderate
NTRU	High	0.9–1.5	2–4	∼1,200	Good
LEDAcrypt	High	>5.0	>40	∼2,000	Poor
HQC	High	2.5–3.0	9–12	∼1,800	Fair
ROLLO (Proposed)	Moderate	0.1–0.2	<5	∼2,089	**Excellent**

communication records cannot be obtained, thereby achieving PFS.

Although direct battery usage was not measured, we estimated energy consumption based on the Jetson Nano's active power draw (5W) and execution time. The full protocol phase (registration + authentication) consumes approximately 0.675 joules on average. These values support the protocol's suitability for energy-constrained IoT devices, though more precise profiling on MCU platforms is planned in future work.

VI. COMPARATIVE STUDY AND STATE-OF-THE-ART

The evolving landscape of quantum computing poses a critical threat to classical cryptographic techniques, especially those widely used in lightweight IoT environments such as Elliptic Curve Cryptography (ECC) and RSA. To address this, several post-quantum cryptographic (PQC) solutions have been proposed, emphasizing the trade-off between quantum resistance, computational efficiency, and suitability for resource-constrained IoT devices.

Comparing candidates for Post-Quantum Cryptography (PQC) focuses on factors such computing efficiency, hardware implementation requirements, security assumptions, and key and signature sizes. For example, CRYSTALS-Kyber (KEM) and CRYSTALS-Dilithium (signature) provide useful functionality, but their key/signature sizes are huge. Based on hash functions, SPHINCS+ provides strong security but comes with a hefty overhead. Because of their complicated matrix operations and high key sizes, other methods, such as McEliece, are less appropriate for environments with limited resources.

Table IX presents a comparative analysis of representative post-quantum schemes evaluated across key performance metrics, including key generation time, encryption/decryption latency, and communication overhead. Notably, NIST finalist algorithms such as CRYSTALS-Kyber and BIKE exhibit strong quantum resilience but tend to incur high key sizes and memory overheads, which limit their deployment on embedded or battery-operated systems. In contrast, schemes like ROLLO and LEDAcrypt offer more lightweight alternatives due to their compact structures and efficient key encapsulation mechanisms.

Recent works such as [9], [10], [13], [16] have focused on incorporating PQC into multimedia systems, deep learning models, and blind signature schemes. However, most existing protocols either lack implementation on real IoT hardware or overlook the communication latency and memory constraints associated with practical deployment. Further, a significant number of studies prioritize lattice-based cryptosystems, which, while secure, often introduce polynomial multiplication and convolution overhead that can be unsuitable for real-time consumer IoT applications [20], [30].

In contrast, the proposed scheme in this manuscript leverages the ROLLO family of code-based cryptography for constructing a lightweight communication protocol tailored for IoT. This choice allows us to maintain low encryption/decryption costs, reduced communication overhead, and minimal memory usage—making the solution ideal for edge and fog environments where computation and power budgets are tight. Moreover, by integrating XOR-based primitives and optimized key encapsulation techniques, our protocol exhibits superior performance in both security and execution time when benchmarked against state-of-the-art implementations.

This comparative analysis highlights a crucial gap in lightweight PQC adoption for real-time, resource-constrained IoT systems. Our proposed approach addresses these limitations by offering a scalable, post-quantum secure protocol with minimal cryptographic overhead, thereby pushing the current boundary of PQC applicability in IoT.

The proposed lightweight post-quantum secure communication protocol, based on ROLLO code-based cryptography, demonstrates strong performance for IoT applications. It achieves significantly lower key generation, encryption, and decryption times compared to other NIST candidate algorithms, particularly outperforming Classic McEliece and LEDAcrypt in terms of efficiency. The protocol effectively balances security and computational cost by relying on lightweight operations such as hashing and XOR, making it highly suitable for energy-constrained IoT devices. Security evaluations confirm resilience against common attack vectors, and the protocol scales efficiently with increasing network sizes, maintaining low communication overhead. Overall, the proposed solution proves to be a practical and secure choice for post-quantum communication in resource-constrained IoT environments.

VII. CONCLUSION

This article proposes a lightweight, post-quantum secure communication protocol optimized for IoT devices operating under strict resource constraints. As traditional cryptographic algorithms like RSA and ECC become vulnerable to quantum attacks, the proposed protocol leverages ROLLO-II—a code-based cryptographic scheme currently under NIST consideration—for its small key sizes and efficient encryption/decryption capabilities.

To maintain a lightweight design, the protocol minimizes expensive cryptographic operations and instead uses efficient

primitives such as SHA-512 hash functions and XOR operations. This significantly reduces computational overhead, making the protocol suitable for real-time, energy-sensitive IoT environments.

Performance evaluations demonstrate strong efficiency on both constrained and high-performance platforms. On a Jetson Nano (ARM), the protocol achieved registration and authentication times of 90 ms and 45 ms, respectively. On an Intel i9 processor, these times were reduced to 5 ms and 2.5 ms. The protocol requires only 3 encryption operations during registration and 2 during authentication, outperforming many existing schemes in cryptographic overhead. In terms of security, the protocol ensures device anonymity and defends against common attacks such as replay and man-in-the-middle. It also achieves perfect forward secrecy by updating passwords each session using unique nonce values. Even if a device is later compromised, past communications remain secure. Overall, the protocol offers a robust, quantum-resistant solution for secure IoT communication, balancing strong cryptographic guarantees with practical efficiency. Future work will focus on enhancing scalability, implementing on microcontroller-class devices, and performing formal security analysis under comprehensive threat models.

REFERENCES

[1] E. Lee, Y.-D. Seo, S.-R. Oh, and Y.-G. Kim, "A survey on standards for interoperability and security in the Internet of Things," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 2, pp. 1020–1047, 2nd Quart., 2021.

[2] M. N. Bhuiyan, M. M. Rahman, M. M. Billah, and D. Saha, "Internet of Things (IoT): A review of its enabling technologies in healthcare applications, standards protocols, security, and market opportunities," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10474–10498, Jul. 2021.

[3] A. Aldosary et al., "A secure authentication framework for consumer mobile crowdsourcing networks," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 4430–4442, May 2025.

[4] S. Hadayeghparast, S. Bayat-Sarmadi, and S. Ebrahimi, "High-speed post-quantum cryptoprocessor based on RISC-V architecture for IoT," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 15839–15846, Sep. 2022.

[5] N. Xiao, Z. Wang, and X. Sun, "A secure and lightweight authentication scheme for digital forensics in Industrial Internet of Things," *Alexandria Eng. J.*, vol. 121, pp. 117–127, May 2025.

[6] T. M. Fernández-Caramés, "From pre-quantum to post-quantum IoT security: A survey on quantum-resistant cryptosystems for the Internet of Things," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6457–6480, Jul. 2020.

[7] Z. Siddiqui, J. Gao, and M. K. Khan, "An improved lightweight PUF–PKI digital certificate authentication scheme for the Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 19744–19756, Oct. 2022.

[8] D. S. C. Putranto, R. W. Wardhani, H. T. Larasati, and H. Kim, "Space and time-efficient quantum multiplier in post quantum cryptography era," *IEEE Access*, vol. 11, pp. 21848–21862, 2023.

[9] K. B. A. Kumar, L. S. Mohith, K. Jain, P. Krishnan, N. Venkatachalam, and R. Buyya, "Post-quantum cryptography based multimedia encryption communication scheme in IoT consumer electronics," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 4995–5006, May 2025.

[10] A. Sharma and S. Rani, "Post-quantum cryptography (PQC) for IoT-consumer electronics devices integrated with deep learning," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 4925–4933, May 2025.

[11] H. Gharavi, J. Granjal, and E. Monteiro, "Post-quantum blockchain security for the Internet of Things: Survey and research directions," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 3, pp. 1748–1774, 3rd Quart., 2024.

[12] M. S. Peelam and V. Chamola, "Enhancing security using quantum blockchain in consumer IoT networks," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 4819–4837, May 2025.

[13] R. Huang and A. Sharma, "Post-quantum verifiable decryption scheme for Internet of Thing based consumer electronic edge device," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 4903–4913, May 2025.

[14] P. Zeng, K.-K. R. Choo, and D.-Z. Sun, "On the security of an enhanced novel access control protocol for wireless sensor networks," *IEEE Trans. Consum. Electron.*, vol. 56, no. 2, pp. 566–569, May 2010.

[15] O. Pal and B. Alam, "Efficient and secure key management for conditional access systems," *IEEE Trans. Consum. Electron.*, vol. 66, no. 1, pp. 1–10, Feb. 2020.

[16] M. E. Seno et al., "Post quantum-resistant blind signature scheme for consumer Internet of Things security," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 4949–4958, May 2025.

[17] J.-M. Yao, Q. Li, H.-K. Mao, and A. A. A. El-Latif, "A practical multi-protocol collaborative quantum key distribution networking scheme for consumer electronics devices," *IEEE Trans. Consum. Electron.*, vol. 71, no. 1, pp. 1190–1200, Feb. 2025.

[18] A. Castiglione, J. G. Esposito, V. Loia, M. Nappi, C. Pero, and M. Polsinelli, "Integrating post-quantum cryptography and blockchain to secure low-cost IoT devices," *IEEE Trans. Ind. Informat.*, vol. 21, no. 2, pp. 1674–1683, Feb. 2025.

[19] O. S. Althobaiti and M. Dohler, "Quantum-resistant cryptography for the Internet of Things based on location-based lattices," *IEEE Access*, vol. 9, pp. 133185–133203, 2021.

[20] M. A. Hafeez, W.-K. Lee, A. Karmakar, and S. O. Hwang, "Efficient TMVP-based polynomial convolution on GPU for post-quantum cryptography targeting IoT applications," *IEEE Internet Things J.*, vol. 11, no. 13, pp. 23428–23443, Jul. 2024.

[21] A. Ahmad and S. Jagatheswari, "Lattice-based three party authenticated key agreement scheme in medical IoT for post-quantum environment," *IEEE Access*, vol. 12, pp. 157247–157259, 2024.

[22] K.-A. Shim, "On the suitability of post-quantum signature schemes for Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 10648–10665, Mar. 2024.

[23] A. Couvreur, I. Márquez-Corbella, and R. Pellikaan, "Cryptanalysis of McEliece cryptosystem based on algebraic geometry codes and their subcodes," *IEEE Trans. Inf. Theory*, vol. 63, no. 8, pp. 5404–5418, Aug. 2017.

[24] E. Kirshanova and A. May, "Breaking Goppa-based McEliece with hints," *Inf. Comput.*, vol. 293, Aug. 2023, Art. no. 105045.

[25] Z. Sun, J. Zhuang, Z. Zhou, and F.-W. Fu, "A new McEliece-type cryptosystem using Gabidulin-Kronecker product codes," *Theor. Comput. Sci.*, vol. 994, 2024, Art. no. 114480. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0304397524000951

[26] J. Hu, W. Wang, K. Gaj, L. Wang, and H. Wang, "Engineering practical rank-code-based cryptographic schemes on embedded hardware. A case study on ROLLO," *IEEE Trans. Comput.*, vol. 72, no. 7, pp. 2094–2110, Jul. 2023.

[27] J. Lablanche, L. Mortajine, O. Benchaalal, P.-L. Cayrel, and N. E. Mrabet, "Optimized implementation of the NIST PQC submission ROLLO on microcontroller," IACR Cryptology ePrint Arch., IACR, Bellevue, WA, USA, Rep. 2019/787, 2019.

[28] M. Hamada, S. A. Salem, and F. M. Salem, "LAMAS: Lightweight anonymous mutual authentication scheme for securing fog computing environments," *Ain Shams Eng. J.*, vol. 13, no. 6, 2022, Art. no. 101752. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2090447922000636

[29] C.-Y. Weng, C.-T. Li, C.-L. Chen, C.-C. Lee, and Y.-Y. Deng, "A lightweight anonymous authentication and secure communication scheme for fog computing services," *IEEE Access*, vol. 9, pp. 145522–145537, 2021.

[30] H.-G. Joo, S. Lee, and D.-J. Shin, "Extended number theoretic transform for lightweight post-quantum cryptosystems in IoT," *IEEE Internet Things J.*, vol. 12, no. 6, pp. 7376–7388, Mar. 2025.